

Diseño de una base de datos relacional

Del modelo entidad-relación a las consultas analíticas

E-commerce de la Tienda de Campus

Universidad del Rosario · Ingeniería de Datos

Un recorrido metodológico: PostgreSQL, normalización y SQL analítico

El caso de estudio

Contexto

Modelar la tienda de campus de la Universidad, un sistema de comercio electrónico al estilo de Amazon o Mercado Libre.

Estudiantes

Usuarios con credencial institucional e información de contacto.

Catálogo

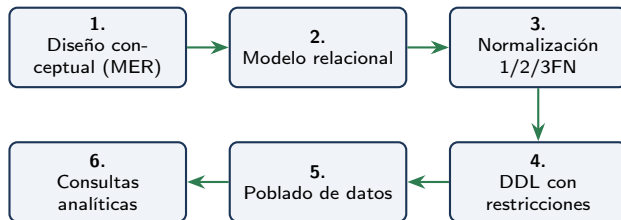
Productos organizados por categorías, con precio y disponibilidad.

Pedidos

Flujo completo: carrito, pago y seguimiento de entregas.

Objetivo: **la menor cantidad de tablas posible**, respetando la normalización.

La hoja de ruta



Cada etapa alimenta a la siguiente: del concepto a los datos reales.

El modelo entidad-relación usa convenciones de forma:

- **Rectángulo** = entidad
- **Rombo** = relación
- **Óvalo** = atributo (subrayado = llave primaria)
- **1, N, M** = cardinalidad

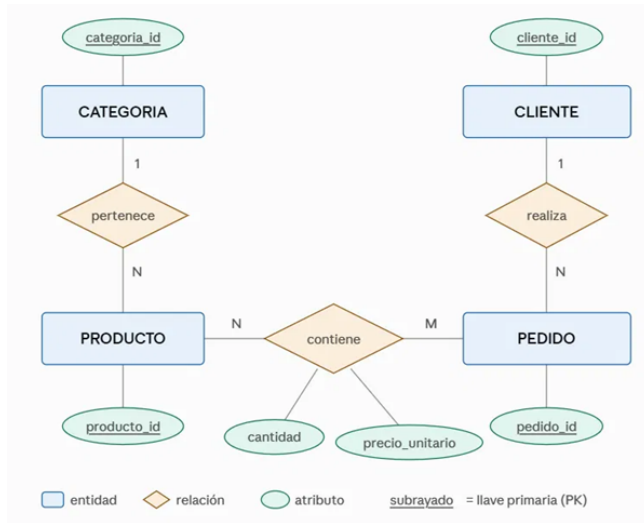
Cuatro entidades bastan

CLIENTE, CATEGORIA, PRODUCTO y PEDIDO.

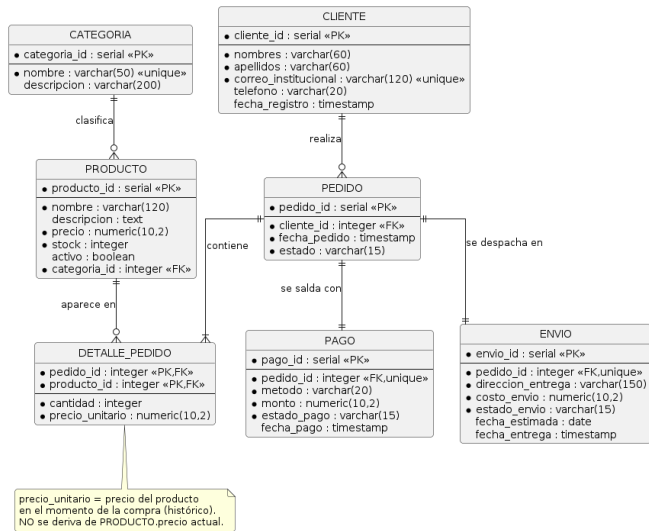
La relación *contiene* (pedido-producto) es de muchos a muchos y carga atributos propios: cantidad y precio unitario.

En el modelo relacional, ese muchos a muchos se convertirá en una tabla puente.

Paso 1 | El MER, visualmente



Paso 2 | Modelo relacional



1FN — Atomicidad

Cada campo guarda un valor único e indivisible. Por eso nombres y apellidos van separados.

2FN — Llave completa

En `detalle_pedido`, cantidad y precio dependen del *par* (pedido, producto), no de una parte.

3FN — Independencia

El nombre de la categoría vive en su tabla. El total del pedido no se almacena: se deriva.

Normalizar aquí significó **restar** redundancias, no sumar complejidad.

Paso 4 | DDL con restricciones

Las reglas de negocio se codifican como PK, FK, UNIQUE y CHECK:

```
CREATE TABLE producto (  
    producto_id SERIAL PRIMARY KEY,  
    nombre      VARCHAR(120) NOT NULL,  
    precio      NUMERIC(10,2) NOT NULL,  
    stock       INTEGER NOT NULL DEFAULT 0,  
    categoria_id INTEGER NOT NULL REFERENCES categoria(categoria_id),  
    CONSTRAINT chk_precio CHECK (precio >= 0),  
    CONSTRAINT chk_stock  CHECK (stock >= 0)  
);
```

La base se defiende sola: rechaza cualquier dato que viole las reglas.

Tablas base — Mockaroo

Datos realistas para cliente (100) y producto (150), exportados a CSV.

Tablas transaccionales — SQL

pedido, detalle_pedido, pago y envio generados con INSERT...SELECT aleatorio.

Tabla	Filas
categoria	8
cliente	100
producto	150
pedido	300
detalle_pedido	≈900
pago / envio	300 / 300

Ingresos por categoría

Categoría	Ingresos
Libros	16.5 M
Alimentos y bebidas	15.9 M
Regalos y souvenirs	14.1 M
Merchandising oficial	13.5 M

Total de cada pedido

Se calcula al vuelo con `SUM()`, uniendo pedido, detalle, envío y pago. Nunca se almacena: es la **3FN en acción**.

Otras vistas

Productos más vendidos, ventas por período, clientes frecuentes.

- Se recorrió el ciclo completo: **concepto** → **modelo** → **datos** → **análisis**.
- Siete tablas normalizadas capturan todo el comercio electrónico sin redundancia.
- Las restricciones garantizan la integridad, sin importar cómo entren los datos.
- Lo que se puede derivar (el total) **no se almacena**: se consulta.

Del diagrama a la consulta: un sistema íntegro y consultable.